AI23531 – DEEP LEARNING

EX 9 TEXT GENERATION USING LSTM NETWORKS

AIM

To build a text-generation model using Long Short-Term Memory (LSTM) networks, train it on a text corpus (Shakespeare) and generate new text sequences.

PROCEDURE

1. Download the Tiny Shakespeare corpus and convert it to lower case.
2. Build character-to-index and index-to-character dictionaries.
3. Create input sequences of 40 characters with the following character as the target.
4. Build the model: Embedding(64) → LSTM(128) → Dense(softmax over characters).
5. Compile with sparse categorical cross-entropy and the Adam optimizer; train for 1 epoch (batch size 128).
6. Define a generation function that predicts one character at a time by sampling from the output probabilities.
7. Generate 300 characters from the seed "shall i compare thee to a summer's day?".

ALGORITHM

1. Start.
2. Load the text and create the character vocabulary.
3. Slice the text into overlapping sequences of length 40 and their next characters; encode them as integers.
4. Train the Embedding–LSTM–Dense model to predict the next character.
5. Take the seed text and convert it to indices.
6. Repeat 300 times: feed the last 40 characters to the model, obtain the probability distribution, sample the next character and append it.
7. Convert the indices back to text and print the result.
8. Stop.

PROGRAM

```
# Step 1: Load and Preprocess the Dataset
import tensorflow as tf
import numpy as np

text = tf.keras.utils.get_file('shakespeare.txt',
    'https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt')
text = open(text, 'r').read().lower()
chars = sorted(set(text))
c2i = {c: i for i, c in enumerate(chars)}
i2c = {i: c for i, c in enumerate(chars)}

# Step 2: Create Input and Output Sequences
seq_len = 40
X = []
y = []
for i in range(len(text) - seq_len):
```

```
    input_seq = text[i:i + seq_len]
    target_char = text[i + seq_len]
    X.append([c2i[c] for c in input_seq])
    y.append(c2i[target_char])

X = np.array(X)
y = np.array(y)

# Step 3: Build the LSTM Model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(len(chars), 64, input_length=seq_len),
    tf.keras.layers.LSTM(128),
    tf.keras.layers.Dense(len(chars), activation='softmax')
])
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam')
model.fit(X, y, batch_size=128, epochs=1)

# Step 4: Define the Text Generation Function
def generate(seed, length=300):
    seq = [c2i[c] for c in seed.lower()]
    for _ in range(length):
        inp = np.array(seq[-seq_len:]).reshape(1, -1)
        pred = model.predict(inp, verbose=0)[0]
        next_idx = np.random.choice(len(pred), p=pred)
        seq.append(next_idx)
    return seed + ''.join(i2c[i] for i in seq[len(seed):])

# Step 5: Generate and Display Text
print("\nGenerated Text:\n")
print(generate("shall i compare thee to a summer's day?\n"))
```

OUTPUT

```
8714/8714 ━━━━━━━━━━━━━━━━━━━━ 1092s 125ms/step - loss: 1.8837

Generated Text:

shall i compare thee to a summer's day?
what hear with there's yourselfies ofk the trosper
in be in a lord, stake you, insered,
nay shall not by wreatly we would sere;
and be did, and jilia

lucio:
you nats and the reads gowsta'tch and not a gonod.

henrirr:
how i wip newy of froat; heart, and, mer!
evend hand, of hear's and much the flor
```

Console output

RESULT

After one epoch (loss 1.8837) the model produced Shakespeare-style text with correct line and speaker-name structure, but many words were invented; more epochs would improve fluency and coherence.